

Document number: P2166R0

Project: Programming Language C++

Audience: LEWG-I, LEWG, LWG

Yuriy Chernyshov <georgthegreat@gmail.com>, <thegeorg@yandex-team.ru>

Date: 2020-05-06

A Proposal to Prohibit `std::basic_string` and `std::basic_string_view` construction from `nullptr`.

Introduction and Motivation

According to the C++ Standard, the behavior of `std::basic_string::basic_string(const CharT* s)` constructor is *undefined* if `[s, s + Traits::length(s))` is not a valid range (for example, if `s` is a null pointer) (citation is taken [from cppreference.com](http://en.cppreference.com), the standard have slightly different wording in [21.3.2.2 \[string.cons\]](#)). Same applies to `std::basic_string_view::basic_string_view(const CharT* s)` constructor.

Existing implementations (i. e. [libc++](#)) might add a runtime assertion to forbid such behavior. Certain OpenSource projects would trigger this assertion. The list includes, but not limited to:

- [poco](#),
- [hdf5](#),
- [llvm](#) project itself, though the code is marked as unreachable.

On a large private monorepo applying proposed changes and running an automatic CI-check helped to find 7 problematic projects (the number includes projects listed above), one of which would actually segfault if the code was reached (and the code was really easy reachable).

This proposal attempts to improve the diagnostics by explicitly deleting the problematic constructors, thus moving these assertions to compile time.

Impact on the Standard

This proposal changes `<string>` and `<string_view>` headers only and does not affect the language core.

Proposed Wording

The wording is relative to [N4861](#).

1. Modify [21.3.2 \[basic.string\]](#) as follows:

```
[...]
namespace std {
    template<class charT, class traits = char_traits<charT>,
            class Allocator = allocator<charT>>
    class basic_string {
    public:
        // types
        [...]
```

```

    // [string.cons], construct/copy/destroy
    [...]
    constexpr basic_string(const charT* s, size_type n, const Allocator& a = Allocator());
    constexpr basic_string(const charT* s, const Allocator& a = Allocator());
+   constexpr basic_string(nullptr_t) = delete;
    [...]
    template<class T>
    constexpr basic_string& operator=(const T& t);
    constexpr basic_string& operator=(const charT* s);
+   constexpr basic_string& operator=(nullptr_t) = delete;
    [...]
};
[...]
```

2. Modify [21.4.1 \[string.view.synop\]](#) as indicated:

```

[...]
```

```

template<class charT, class traits = char_traits<charT>>
class basic_string_view {
public:
    // types
    [...]
    constexpr basic_string_view(const charT* str);
+   constexpr basic_string_view(nullptr_t) = delete;
    [...]
};
```

Further Discourse

These changes would not allow to remove runtime check, the following code will remain compilable and will trigger the assertion:

```

const char *p = nullptr; // or more likely, p = functionThatCanReturnNull()
string s(p, 3);
```

As a development of the above proposal it seems logical to remove sized counterpart of nullptr constructors, as *the behavior is undefined if $[s, s + \text{count})$ is not a valid range* (citation source is the same). That is, the following statements are suggested where appropriate:

```

basic_string(nullptr_t, size_t) == delete;
constexpr basic_string_view(nullptr_t, size_t) == delete;
```

These changes will break the legal, yet not legitimate case of constructing `std::string` using `basic_string(nullptr, 0)`; and `std::string_view` using `basic_string_view(nullptr, 0)`; and thus they were not included into the main text of the proposal.

Acknowledgements

The author would like to thank Antony Poloukhin, Marshall Clow and Eric Fiselier for a thorough review and suggestions.